

Map2Route: Benchmarking Compositional Language-Grounded Route Planning over Semantic Maps

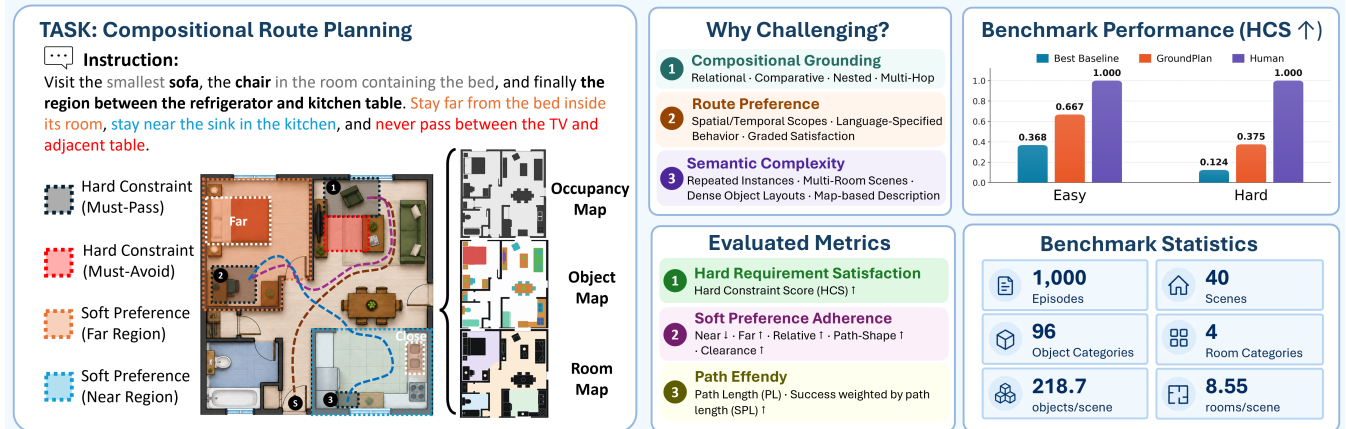


Fig. 1. Overview of Map2Route. Given a pre-built semantic map and a natural-language instruction, the task is to generate a complete route that resolves compositional references and follows ordered, scoped route requirements and preferences. Map2Route contains 1,000 episodes across 40 multi-room scenes and evaluates performance across hard constraints, soft-preference adherence, and path efficiency. Existing methods remain substantially below human performance, highlighting the challenge of compositional language-grounded route planning.

Abstract—We introduce **Map2Route**, a human-curated benchmark for compositional language-grounded route planning over pre-built semantic maps. Map2Route contains 1,000 episodes across 40 scenes, where instructions use relational, comparative, and nested descriptions to identify route-relevant objects and regions, while specifying ordered must-pass regions, must-avoid requirements, five categories of soft preferences, and spatial and route-stage scopes, which is partially tested by existing works. Alongside Map2Route, we propose **Grounding2Route**, which combines executable code-as-grounding with verification-guided repair and scope-aware planning. Across seven representative adapted baselines, **Grounding2Route** substantially outperforms existing methods in all metrics. Despite these gains, a substantial gap to human demonstrations remains, highlighting the difficulty of Map2Route and the considerable headroom for future progress. Additional qualitative results and resources are available on Project Page.

I. INTRODUCTION

This work considers **language-grounded route planning over pre-built semantic maps**. Given a semantic map containing traversability, object, and room information, and a natural-language instruction, the goal is to generate a complete route that faithfully realizes the instruction on the map. Unlike conventional map-based planning, where destinations and costs are explicitly specified through coordinates or predefined objectives, here the instruction itself describes *where the robot should start, go, and how it should navigate*. This formulation enables users to command robots directly in map-based natural language. For example, in a household environment, a user may ask a robot to “go to the chair nearest to the bed in the bedroom with two windows that is farthest from the kitchen”, describing the destination directly

through natural language rather than specifying target object IDs or exact coordinates of the goal points in the map.

Such language-grounded route planning raises three key challenges. **Challenge 1: Compositional grounding.** The agent must determine which objects or regions in the map are referred to by natural language. Since multiple instances of the same category often exist, targets can be specified through relational, comparative, and nested references. For example, “the chair nearest to the bed in the bedroom with two windows that is farthest from the kitchen” requires several reasoning steps to identify a single chair. **Challenge 2: Soft-preference adherence.** Instructions describe not only *where to go*, but also *how to get there* through soft route preferences. For example, “stay far from the windows while crossing the bedroom” requires the agent to understand the intended preference and generate a route that reflects it. **Challenge 3: Semantic Complexity.** The agent must integrate grounded references and route requirements under substantial semantic complexity, including multi-room traversal, repeated object instances, multiple destinations (e.g., “go to the sofa, then the desk”), ordering constraints (e.g., “before going to the desk, visit the chair”), and conditional branching (e.g., “if the bedroom contains a lamp, go to the lamp; otherwise, go to the table”).

Existing navigation and planning benchmarks cover only subsets of these challenges and even overlook some challenges entirely. To evaluate them jointly, we introduce **Map2Route**, a human-curated benchmark for compositional language-grounded and constraint-aware route planning over semantic maps. Map2Route evaluates each route along three

complementary dimensions: *hard-requirement satisfaction* (*Challenges 1 and 3*), which measures whether a method can correctly resolve compositional references and satisfy ordered, multi-stage route requirements; *soft-preference adherence* (*Challenge 2*), which evaluates whether the generated route follows the soft preferences within specified scopes; and *path efficiency*, which measures how efficiently the complete route fulfills these requirements.

Alongside the benchmark, we propose **Grounding2Route**, a language-to-route framework that separates compositional semantic grounding from geometric planning. Grounding2Route uses executable *code-as-grounding* with verification-guided repair to produce a *Route Semantic Intermediate Representation (RouteIR)*, which is deterministically compiled into a route through scope-aware planning.

We evaluate Map2Route with seven adapted representative methods, Grounding2Route, shortest-feasible A*, and human demonstrations. The strongest baseline satisfies only 29.4% of the *hard-requirement satisfaction*, whereas Grounding2Route improves this to 57.9%, while also outperforming all adapted baselines across all metrics. Despite these gains, the substantial gap to human demos shows that Map2Route remains highly challenging and far from saturated.

Our contributions are threefold:

- We introduce **Map2Route**, a human-curated benchmark for compositional language-grounded route planning over semantic maps.
- We propose **Grounding2Route**, a two-stage structured language-to-route framework.
- We provide a comprehensive empirical study across seven adapted baselines, controlled ablations, difficulty analyses, and real-world trials, revealing that the substantial gaps in compositional grounding and constraint-aware planning, and identifying which components are most critical to this task.

II. RELATED WORK

A. Language-Guided Planning over Pre-built Maps

Existing methods address different aspects of language-guided planning over structured spatial representations. VLMaps [1] grounds language-specified goals and spatial relations onto semantic maps, enabling navigation to relationally defined locations. SayPlan and OSG-LLM [2], [3] reason over structured scene representations to identify task-relevant entities and produce task-level or symbolic plans. Temporal-logic methods [4], [5], [6], [7] translate natural-language instructions into formal spatial, temporal, or ordering specifications, while BehAV [8] and NORM-Nav [9] incorporate language-specified behavioral constraints into geometric navigation objectives.

However, these formulations emphasize different subsets of the full route-planning problem. VLMaps [1] primarily focuses on grounding goals and spatial relations, while SayPlan [2] and OSG-LLM [3] emphasize task-level reasoning and action sequencing rather than detailed route preferences. Temporal-logic approaches [4], [5], [6], [7] are well suited

to discrete hard requirements such as ordering or avoidance, but are less naturally suited to graded preferences over route geometry. Conversely, BehAV [8] and NORM-Nav [9] model behavioral preferences during navigation, but do not jointly address complex compositional grounding and multi-stage route semantics. In comparison, Map2Route jointly evaluates all compositional grounding, multi-stage route requirements, and soft preferences in a unified complete-route setting.

B. Online Language-Guided Navigation

Online language-guided navigation requires an agent to interpret language while sequentially interacting with an environment from egocentric observations. Vision-and-Language Navigation (VLN) [10], [11], [12] requires agents to follow natural-language route descriptions, while ObjectNav, REVERIE, and SOON [13], [14], [15], [16] specify navigation goals through semantic categories or referring expressions. These tasks study online perception, grounding, and sequential decision making without access to a pre-built semantic map. Map2Route instead assumes that a semantic map is already available and studies how free-form language can be translated into a complete route over this map.

III. MAP2ROUTE

A. Task Formulation

Each episode provides a semantic map $\mathcal{M} = (\mathcal{M}_{\text{trav}}, \mathcal{M}_{\text{obj}}, \mathcal{M}_{\text{room}})$ comprising aligned traversability, object, and room layers, an initial robot position p_0 , and a natural-language instruction I , but no explicit goal coordinates. The task is to generate a valid route

$$(\mathcal{M}, p_0, I) \longrightarrow \tau = (p_0, p_1, \dots, p_T) \quad (1)$$

that faithfully realizes I in $\mathcal{M}$. Map2Route therefore evaluates the complete generated route.

1) *Semantic Map Representation*: Each environment provides aligned traversability, object, and room maps at 0.05 m/grid resolution, generated offline from ProcTHOR [17] annotations and AI2-THOR [18] metadata. Traversability is obtained from reachable positions, object footprints from projected 3D bounding boxes, and room labels from rasterized room polygons. Walls are rasterized separately and non-square maps are padded as untraversable.

2) *Compositional Route Instructions*: Each instruction is decomposed into hard requirements and soft preferences. Hard requirements include ordered must-pass and must-avoid regions, while soft preferences have five types: Near, Far (be close to or far away from a reference object), Relative (be closer to one object than another), Path-Shape (follow a path of a certain shape), and Clearance (keep distance from obstacles) preferences, each defined with an active region. Language-specified constraints may further have spatial or temporal scopes; Clearance is applied globally by default. This decomposition is used only for annotation and evaluation, while methods receive the original instruction.

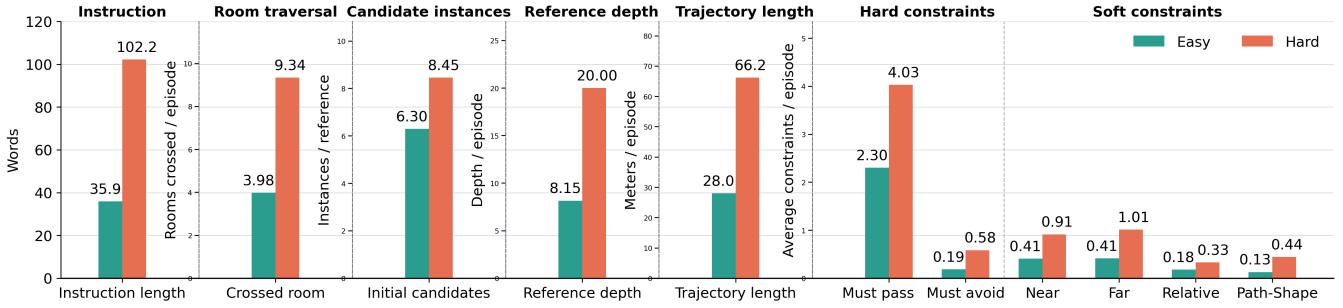


Fig. 2. Complexity statistics of the Easy and Hard subsets in Map2Route.

B. Benchmark Construction

Map2Route contains 1,000 evaluation-only episodes across 40 ProcTHOR-10K [17] scenes, including 700 *Easy* and 300 *Hard* episodes, with 25 instructions per scene. We uniformly subsample 40 scenes from the 120 environments with the largest floor areas and use four additional disjoint scenes for a separate 50-episode development set. Two experts spent approximately 120 hours on annotation and cross-checking, followed by 20 hours of quality control for ambiguity, uniqueness and feasibility. Example episodes are available on the Project Page.

Annotation Principles. We follow three principles: (1) *Complexity.* *Easy* episodes contain 2–4 constraints, while *Hard* episodes contain 5–8 and are annotated afterward to increase compositional complexity. (2) *Diversity.* We encourage diverse objects, rooms, referring expressions, constraint types and combinations, and spatial or route-stage scopes. The first two principles make *Hard* episodes systematically more challenging than *Easy* ones, with more constraints, more complex referring expressions and reference chains, and richer combinations of constraint types and scopes. (3) *Validity.* Each reference must uniquely identify a concrete object or region, and all specified routes must be feasible.

Benchmark Statistics. Fig. 2 summarizes instruction length, room traversal, candidate instances before disambiguation, reference depth, trajectory length, and the numbers of hard and soft constraints. The benchmark further spans 96 object categories and 4 room types. Each scene contains an average of 218.7 object instances and 8.55 rooms. *Hard* episodes show greater complexity than *Easy* episodes across all reported dimensions.

C. Benchmark Metrics

We evaluate along three complementary dimensions: 1) hard-requirement score, which measures whether a method can resolve and satisfy language-specified destinations and ordered route requirements; 2) soft-preference adherence, which evaluates whether the route follows specified preferences within their intended scopes; and (3) path efficiency, which measures the efficiency of the generated route.

1) *Hard Constraint Score (HCS):* A path can be considered containing an ordered list $\mathcal{G}$ of hard requirement regions $G_1, \dots, G_{M+N}$, including M must-pass and N must-avoid region. These regions are specified either through language description (ex. area between the bed and the desk), or

through a reference object. For a reference object O_i ,

$$G_i = \{p \in R(O_i) \mid \|p - c_{O_i}\|_2 \leq r_{O_i} + 1\text{ m}\}, \quad (2)$$

where $R(O_i)$ is its room and c_{O_i}, r_{O_i} denote its footprint center and radius. A region is reached when the route intersects it. HCS is defined as the ratio of the number of correctly reached must-pass regions under the order defined in $\mathcal{G}$, K , and the total number of must-pass regions, M :

$$\text{HCS} = \frac{K}{M}. \quad (3)$$

Note that a route-stage violation stops further credit, while a global violation yields $\text{HCS} = 0$.

2) *Soft Preference Metrics:* Let $\mathcal{W}_R(\tau)$ denote the ordered sequence of distinct route locations within active region R , where repeated visits to the same location are counted only once while preserving the order of first occurrence; $\mathcal{W}(\tau)$ is defined analogously over the full route. Let $d(p, O)$ and $d(p, \mathcal{U})$ denote the closest-point distance between a point p to an object and the nearest non-traversable area, respectively. We select these five preference types because they cover a broad range of common route-shaping preferences, directly affect the geometry of the generated route, and remain sufficiently distinct from one another to avoid highly redundant evaluation. Table I defines the corresponding metrics. Since they capture different route preferences and have different scales and optimization directions, we report them separately rather than combining them into a single score. For each preference type, we first average valid constraint-level measurements within each episode and then macro-average across valid episodes.

For Relative, A is the reference the route should stay closer to and B is the one the route should stay farther from, with mean distances D_A and D_B . Path-Shape uses tolerance-aware nDTW against a canonical trajectory τ_{ref} ($\delta = 0.5\text{ m}$, $d_{\text{succ}} = 0.2\text{ m}$). Independently drawn human trajectories score 0.99 on both Easy and Hard, showing that the tolerance-aware metric assigns nearly identical scores to alternative valid paths and thus supports using a single canonical reference. For Near, Far, Relative, and Path-Shape, failure to enter the target region or complete the relevant segment receives the metric-specific worst value and remains in the denominator; Clearance is always evaluated globally.

Grounding2Route: Executable grounding, verified route semantics, and scope-aware planning over semantic maps

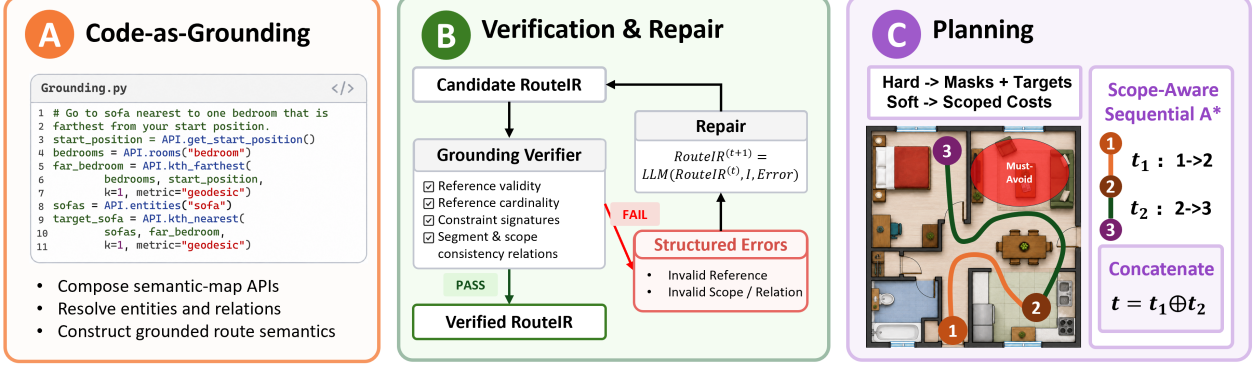


Fig. 3. Overview of the Grounding2Route pipeline. Grounding2Route first uses executable code-as-grounding to resolve compositional route semantics over the semantic map, then verifies and repairs the resulting RouteIR, and finally compiles the verified specification into a complete route through deterministic scope-aware sequential planning.

TABLE I
DEFINITION OF SOFT CONSTRAINT METRICS.

Preference	Metric	Better
Near	$\frac{1}{ \mathcal{W}_R(\tau) } \sum_{p \in \mathcal{W}_R(\tau)} d(p, O)$	↓
Far	$\frac{1}{ \mathcal{W}_R(\tau) } \sum_{p \in \mathcal{W}_R(\tau)} d(p, O)$	↑
Relative	$\frac{D_B}{D_A + \epsilon}$	↑
Path-Shape	$\exp\left(-\frac{\text{DTW}_\delta(\mathcal{W}_R(\tau), \tau_{\text{ref}})}{d_{\text{succ}} \tau_{\text{ref}} }\right)$	↑
Clearance	$\frac{1}{ \tau } \sum_{p \in \tau} d(p, \mathcal{U})$	↑

3) *Path Efficiency*: We report path length (PL) and Success weighted by Path Length (SPL),

$$\text{SPL} = \frac{1}{M} \sum_{m=1}^M S_m \frac{L_m^*}{\max(L_m^*, L_m)}, \quad (4)$$

where $S_m = 1$ if the agent ever reaches G_m from G_{m-1} and 0 otherwise, L_m is the agent's route length starting from G_{m-1} until it reaches G_m (if ever), and L_m^* is the shortest feasible route produced by A* satisfying the same hard constraint connecting G_{m-1} and G_m .

IV. GROUNDING2ROUTE

We propose **Grounding2Route**, a structured language-to-route planning framework, as shown in Fig. 3. Grounding2Route consists of two stages: *grounding* and *route planning*. In the grounding stage, an LLM composes executable read-only semantic-map APIs through *code-as-grounding* (Sec. IV-A), followed by verification-guided repair (Sec. IV-B) to produce a grounded RouteIR (Sec. IV-B). The verified RouteIR is then deterministically compiled into a complete route through scope-aware sequential planning (Sec. IV-D).

A. Executable Code-as-Grounding

Given an instruction I and semantic map $\mathcal{M}$, compositional references may require multi-step reasoning over

relational, comparative, and nested descriptions. Inspired by Code-as-Policies [19], we propose *code-as-grounding* (CaG), where an LLM composes predefined read-only semantic-map APIs and helper logic into a grounding program P , demoed in Fig. 3.A. Executing P produces a provisional grounding

$$\text{RouteIR} = P(\mathcal{M}, p_0), \quad (5)$$

where *RouteIR* is a provisional route semantic intermediate representation, detailed in Sec. IV-B. All map-dependent bindings in G are computed during execution rather than directly predicted by the LLM.

The API provides primitives across six categories: scene/room queries, selection/set operations, ranking/spatial predicates, logic/comparison, region/waypoint construction, and route construction. These primitives can be compositionally combined to resolve nested references. For Path-Shape, the generated code constructs ordered waypoints. Although Path-Shape is evaluated as a soft preference, Grounding2Route enforces the waypoint order during planning because additive spatial costs cannot reliably preserve topological ordering.

The generated program is sandboxed: it may compose exposed primitives through pure helper functions, but cannot inspect raw grid cells, introduce arbitrary absolute coordinates, access planner costs or search parameters, or output a trajectory directly. This separation gives the LLM compositional flexibility while keeping map-dependent entity selection and geometry within a deterministic interface.

B. Route Semantic Intermediate Representation

Existing language-to-planning methods often use LTL [4], [5], [7], but Map2Route requires heterogeneous grounded route semantics and explicit scopes. We therefore introduce the *Route Semantic Intermediate Representation* (RouteIR):

$$\text{RouteIR} = (\mathcal{S}, \mathcal{C}) \quad \mathcal{S} = (S_1, \dots, S_J) \quad \mathcal{C} = (C_1, \dots, C_L) \quad (6)$$

where $\mathcal{S}$ contains ordered grounded destinations, and $\mathcal{C}$ the grounded route requirements. Each constraint is

$$C_i = (\phi_i, \sigma_i), \quad \sigma_i = (Q_i^{\text{tmp}}, Q_i^{\text{spa}}), \quad (7)$$

where Q_i^{tmp} and Q_i^{spa} specify its route-stage and spatial scopes, respectively. The grounded semantic content follows

$$\phi_i \in \{ \text{Avoid}(R), \text{Near}(R), \text{Far}(R), \\ \text{Relative}(R_a, R_b), \text{PathShape}(W_1, \dots, W_m) \}. \quad (8)$$

Here, R denotes a grounded entity or region, and $(W_1, \dots, W_m)$ an ordered waypoint sequence. Clearance is handled separately as a global planner regularizer.

C. Grounding Verification and Bounded Repair

Execution alone does not guarantee that the resulting specification is structurally valid or consistent with the semantic map. Grounding2Route therefore verifies reference validity and cardinality, constraint signatures, segment and scope consistency, and map-dependent relations by recomputing the corresponding queries through the semantic-map APIs. These checks detect execution, structural, and map-consistency errors, but do not independently certify semantic equivalence between the generated program and the original instruction.

When a verification error is detected, the verifier returns structured feedback $E^{(r)}$ for bounded LLM repair:

$$P^{(r+1)} = f_{\text{LLM}}(I, P^{(r)}, E^{(r)}). \quad (9)$$

This allows LLM to reconsider the complete grounding program against the instruction I while correcting the detected error. The repaired program is re-executed and re-verified until verification succeeds or the repair budget is exhausted.

D. Scope-Aware Sequential Route Planning

Given the verified RouteIR, Grounding2Route plans each route segment sequentially using deterministic cost-aware A*. Going from S_{k-1} to S_k , Path-Shape waypoints, and the grounded destination form an ordered target sequence $\{M_{k,1}, \dots, M_{k,m_k}\}$, with

$$\tau_{k,j} = A^*(M_{k,j-1}, M_{k,j}), \quad (10)$$

where consecutive trajectories are concatenated to form the complete route.

Hard avoidance requirements define segment-specific traversability masks, while Near, Far, and Relative preferences are converted into spatial costs. A constraint C_i is active at segment k and location p only within its grounded route-stage and spatial scopes. The transition cost is

$$c_k(p_t, p_{t+1}) = c_{\text{geom}}(p_t, p_{t+1}) + \sum_{i=1}^L \lambda_{\phi_i} a_i(k, p_{t+1}) J_{\phi_i}(p_{t+1}) \\ + \lambda_{\text{clear}} J_{\text{clear}}(p_{t+1}), \quad (11)$$

$$a_i(k, p) = \mathbf{1}[Q_i^{\text{tmp}} = \emptyset \vee k \in Q_i^{\text{tmp}}] \\ \cdot \mathbf{1}[Q_i^{\text{spa}} = \emptyset \vee p \in Q_i^{\text{spa}}]. \quad (12)$$

where a_i is the indicator, c_{geom} is the standard 8-connected step cost, λ is weight for each soft preferences.. We use $J_{\text{near}}(p) = \text{clip}(d_{R_i}(p)/r_{\text{near}}, 0, 1)$, $J_{\text{far}}(p) = \exp(-d_{R_i}(p)/r_{\text{far}})$, $J_{\text{relative}}(p) = d_{\text{close}}(p)/(d_{\text{close}}(p) + d_{\text{far}}(p) + \varepsilon)$, and $J_{\text{clear}}(p) = ([r_{\text{clear}} - d_{\text{obs}}(p)] + /r_{\text{clear}})^2$.

We set $r_{\text{near}} = r_{\text{far}} = 1.5, \text{m}$ and $r_{\text{clear}} = 0.75, \text{m}$. Clearance is applied globally.

Here, d_{R_i} is the distance to reference region R_i , d_{close} and d_{far} to the closer/farther references, and d_{obs} to the nearest obstacle. All weights and distance scales are fixed across scenes. If any stage is infeasible, planning terminates without modifying RouteIR or triggering further LLM repair.

V. EXPERIMENT RESULT

We conduct experiments on Map2Route to answer four questions: **(1)** How challenging is compositional language-grounded route planning for existing methods, and how does Grounding2Route compare with representative baselines (Section V-B)? **(2)** Which Grounding2Route design (controlled and task-specific adaptations) choices are critical to performance (Section V-C)? **(3)** What factors contribute to the difficulty of Map2Route, and how does performance vary with benchmark complexity (Section V-D)? **(4)** Can Grounding2Route execute compositional language-specified routes in real-world environments (Section V-E)?

A. Experiment Setup

Evaluation protocol: We evaluate all methods on the full Map2Route benchmark and report results separately on *Easy* and *Hard* episodes. We use Gemini-3.5-Flash for all LLM-based methods. For Grounding2Route, we set $(\lambda_{\text{near}}, \lambda_{\text{far}}, \lambda_{\text{relative}}, \lambda_{\text{clear}}) = (8, 96, 64, 36)$, tuned on the development set and fixed for all benchmark evaluations.

Real-World Experiments: We follow the SysNav [20] pipeline to construct the semantic map. Experiments are conducted on a Mecanum-wheeled mobile robot [21] equipped with a Livox Mid-360 LiDAR and a Ricoh Theta Z1 panoramic camera. A local collision-avoidance planner and a path follower are used to execute the globally planned route. The system runs on a laptop equipped with an Intel [CPU] and an RTX4090.

Baseline adaptation: We compare Grounding2Route with seven representative methods: Lang2LTL [4], SayPlan [2], OSG-LLM [3], Lang2LTL-2 [5], LIMP [6], LTLCodeGen [7], and ILN [22]. Since no prior method natively supports the full Map2Route formulation, we adapt each method while preserving its original reasoning representation and planning pipeline. All methods receive the same benchmark-provided semantic map, traversability grid, instruction, and start pose, while retaining their native map abstractions and planners. This comparison reflects performance under the unified Map2Route setting rather than direct performance on the methods' original tasks.

B. Main Benchmark Performance

Table II compares seven baselines and Grounding2Route with shortest-feasible A* and human demonstrations. The best baseline achieves only 0.368/0.124 HCS on Easy/Hard, revealing substantial difficulty in resolving compositional references. Soft constraints are similarly challenging, with best Path-Shape scores of only 0.02/0.01 versus 1.00 for

TABLE II
PERFORMANCE COMPARISON ON MAP2ROUTE ACROSS EASY AND HARD DIFFICULTY LEVELS.

Method	HCS $\uparrow$	Soft Constraint Metric					Path Efficiency	
		Near $\downarrow$	Far $\uparrow$	Relative $\uparrow$	Path-Shape $\uparrow$	Clearance $\uparrow$	PL	SPL $\uparrow$
Easy								
Shortest Feasible A*	1.000	3.13	2.30	1.10	0.05	0.53	28.0	1.00
Human Demonstration	1.000	1.72	3.71	2.57	0.99	0.73	37.1	0.77
Lang2LTL [4] [CoRL23]	0.300	4.41	0.76	0.49	0.01	0.51	30.1	0.28
SayPlan [2] [CoRL23]	0.102	4.74	0.35	0.13	0.00	0.52	20.2	0.08
OSG-LLM [3] [ICRA24]	0.368	4.66	0.93	0.57	0.02	0.46	21.6	0.36
Lang2LTL-2 [5] [IROS24]	0.248	4.45	0.63	0.42	0.02	0.51	31.3	0.22
LIMP [6] [ICRA25]	0.050	4.90	0.19	0.11	0.00	0.48	11.8	0.04
LTLCodeGen [7] [IROS25]	0.274	6.03	0.65	0.38	0.02	0.49	26.2	0.26
ILN [22] [IROS25]	0.063	4.76	0.24	0.07	0.00	0.64	21.2	0.06
Grounding2Route (Ours)	0.667	3.21	2.76	1.27	0.16	0.74	31.2	0.53
Δ vs. best baseline	+0.299	-1.20	+1.83	+0.70	+0.14	+0.10	-	+0.17
Hard								
Shortest Feasible A*	1.000	2.99	2.41	1.24	0.05	0.53	66.2	1.00
Human Demonstration	1.000	1.80	3.59	2.21	0.99	0.73	88.2	0.76
Lang2LTL [4] [CoRL23]	0.124	4.61	0.41	0.16	0.00	0.51	72.7	0.10
SayPlan [2] [CoRL23]	0.029	4.83	0.14	0.05	0.00	0.52	44.8	0.03
OSG-LLM [3] [ICRA24]	0.120	5.56	0.38	0.15	0.00	0.35	31.8	0.11
Lang2LTL-2 [5] [IROS24]	0.094	4.68	0.29	0.09	0.01	0.51	53.7	0.08
LIMP [6] [ICRA25]	0.012	4.96	0.05	0.00	0.00	0.43	13.2	0.01
LTLCodeGen [7] [IROS25]	0.101	6.04	0.34	0.16	0.01	0.45	57.6	0.09
ILN [22] [IROS25]	0.011	4.91	0.07	0.02	0.00	0.65	19.0	0.01
Grounding2Route (Ours)	0.375	3.33	2.24	0.89	0.08	0.69	54.7	0.29
Δ vs. best baseline	+0.251	-1.28	+1.83	+0.73	+0.07	+0.04	-	+0.18

humans, demonstrating the limited ability of current methods to satisfy soft constraints.

Grounding2Route substantially improves performance across all three evaluation dimensions, achieving 0.667/0.375 HCS and 0.53/0.29 SPL on Easy/Hard. It also outperforms all seven baselines across the five soft-preference metrics, showing stronger soft-constraint satisfaction. Despite these gains, a large gap to human performance remains: on Hard, Grounding2Route achieves 0.375 HCS and 0.08 Path-Shape, compared with 1.00 and 0.99 for human demonstrations.

C. Method Ablation Study

Unlike the minimally adapted methods in the main benchmark comparison, the following ablations use controlled, task-specific variants to isolate individual design choices in Grounding2Route. Table III evaluates each key component design choices while keeping the remaining pipeline fixed.

1) *Grounding strategy*: We compare CaG with two direct-prediction variants and an iterative tool-calling variant. *Schema* predicts a schema-constrained JSON intent graph that is deterministically compiled for grounding, while *Direct ID* selects referenced entity IDs from a simplified map representation. *ToolCall* uses the same semantic APIs and downstream pipeline as CaG, but invokes the APIs iteratively through LLM function calls instead of generating an executable grounding program. Schema, Direct ID, and ToolCall achieve HCS/SPL of 0.338/0.27, 0.474/0.39, and 0.539/0.44, respectively, compared with 0.579/0.46 for CaG. This controlled comparison shows that executable program

composition is more effective than iterative tool calling for compositional route grounding.

2) *Intermediate representation*: Replacing RouteIR with LTL reduces HCS/SPL from 0.579/0.46 to 0.439/0.36, showing that RouteIR is better suited to representing the heterogeneous route semantics and scopes in Map2Route.

3) *Verification and repair*: We vary the maximum number of verification-guided repair rounds. Increasing the budget progressively improves HCS from 0.395 at $R = 0$ to 0.506, 0.566, and 0.579 at $R = 1, 2, 3$, respectively. We also observe improvements across all five soft-constraint metrics, with Relative increasing from 0.56 to 1.11. These gains indicate that repair corrects not only hard requirements but also the grounding of soft preferences and their scopes.

4) *Soft-constraint modeling*: We ablate both explicit scope modeling defined in RouteIR and preference-aware planning. We first globalize the temporal scope (Global-T), spatial scope (Global-S), or both (Global-TS), such that constraints remain active throughout the full route, the full map, or both. These variants leave HCS nearly unchanged but reduce Far from 2.52 to 1.85–1.91 and Relative from 1.11 to 0.84–0.87. We further remove all soft-preference costs while retaining the grounded destinations and hard constraints. This produces higher path efficiency but reduces all soft metrics. These results show explicitly modeling both soft preferences and their scopes is essential for language-faithful route planning.

5) *Global versus sequential planning*: We replace sequential planning with a global variant that jointly optimizes

TABLE III

ABLATION STUDY OF GROUNDING2ROUTE ON MAP2ROUTE. EACH ROW MODIFIES ONE COMPONENT OF THE FULL GROUNDING2ROUTE CONFIGURATION WHILE KEEPING ALL REMAINING COMPONENTS FIXED.

Grounding2Route Configuration				HCS $\uparrow$	Soft Preference Metrics					Path Efficiency	
Grounding	IR	Repair	Soft Cons.		Near $\downarrow$	Far $\uparrow$	Relative $\uparrow$	Path-Shape $\uparrow$	Clearance $\uparrow$	PL $\downarrow$	SPL $\uparrow$
CaG	RouteIR	3	Full	0.579	3.27	2.52	1.11	0.11	0.72	38.2	0.46
Schema	RouteIR	3	Full	0.338	4.25	1.05	0.38	0.06	0.60	19.6	0.27
Direct ID	RouteIR	3	Full	0.474	3.91	1.45	0.84	0.01	0.69	28.2	0.39
ToolCall	RouteIR	3	Full	0.539	3.84	1.67	0.84	0.02	0.66	27.2	0.44
CaG	LTL	3	Full	0.439	3.92	1.10	0.70	0.03	0.65	24.3	0.36
CaG	RouteIR	0	Full	0.395	4.21	1.15	0.56	0.05	0.61	21.3	0.32
CaG	RouteIR	1	Full	0.506	4.82	1.58	0.74	0.08	0.66	30.7	0.41
CaG	RouteIR	2	Full	0.566	3.69	1.79	0.84	0.08	0.71	36.2	0.45
CaG	RouteIR	3	Global-T	0.577	3.70	1.85	0.85	0.06	0.71	42.0	0.45
CaG	RouteIR	3	Global-S	0.580	3.67	1.91	0.87	0.10	0.71	37.9	0.47
CaG	RouteIR	3	Global-TS	0.577	3.70	1.87	0.84	0.07	0.70	41.1	0.46
CaG	RouteIR	3	Hard Only	0.581	3.79	1.36	0.58	0.04	0.48	31.2	0.56



Fig. 4. Qualitative real-world evaluation of Grounding2Route. Two representative episodes demonstrate multi-stage target grounding and execution under language-specified route constraints, including scoped far preferences and must-avoid requirements. The executed trajectories and semantic maps demonstrate successful execution of compositional route instructions indoors.

waypoint selection and paths across all segments. Global planning improves SPL by only 0.4%, while increasing the average planning time from 1.04s to 11.06s, a 10.64 $\times$ slowdown. The gain is limited because route quality is dominated by inter-region traversal, while jointly optimizing the exact waypoint within each target region provides only marginal additional benefit. We therefore adopt sequential planning for a better performance–efficiency trade-off.

6) Planner over Oracle
Grounding: Since the soft metrics are coupled with HCS to avoid metric hacking, we provide ground-truth RouteIR to isolate our scope-aware planner. From Table IV, all targets are reached, and our planner attains

TABLE IV
 PLANNER ABLATION OVER ORACLE GROUNDING.

Planner	Near $\downarrow$	Far $\uparrow$	Rel. $\uparrow$
Shortest A*	3.07	2.35	1.16
Human Demo	1.76	3.65	2.42
Ours	2.09	3.33	2.06

2.09/3.33/2.06 on Near/Far/Relative, consistently improving over shortest-path A* and recovering 74% of the A*-to-human gap on average. These results suggest that grounding remains the dominant source of error in the full pipeline, while a smaller but non-negligible gap also remains in soft-constraint-aware planning.

D. Benchmark Difficulty Analysis

Fig. 5 analyzes three complementary sources of benchmark difficulty: compositional reference depth, instance ambiguity, and hard-constraint complexity. HCS decreases for both Grounding2Route and OSG-LLM as each factor increases, with larger degradation under deeper references and more hard constraints. Grounding2Route remains consistently stronger across all complexity levels, but also declines in the hardest settings, highlighting the increasing difficulty

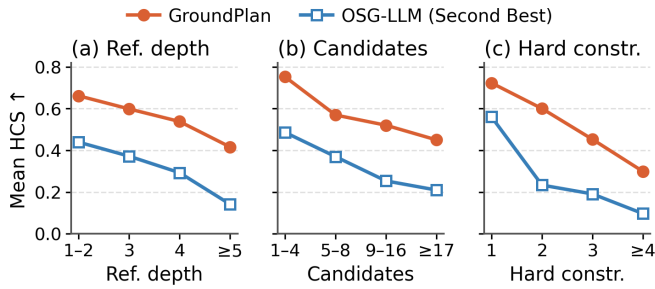


Fig. 5. Difficulty analysis of Map2Route. Mean HCS of Grounding2Route and OSG-LLM across increasing reference depth, candidate instances and hard constraint number.

of compositional grounding and route reasoning.

E. Real-World Experiments

We further evaluate Grounding2Route in the real world with 48 episodes across 8 indoor scenes. Grounding2Route achieves 72.22% HCS. Among the HCS loss, 73.75% result from semantic-map construction errors, 22.5% from Grounding2Route failures in grounding or route planning, and 3.75% from SLAM issues. Qualitative results in Fig. 4 and Project Page show Grounding2Route’s ability to achieve soft preferences during real-world navigation.

VI. CONCLUSION

We introduced Map2Route, a benchmark for compositional language-grounded route planning over semantic maps, covering nested references, ordered hard requirements, and scoped soft preferences. Evaluation with seven adapted baselines and human demonstrations shows that current methods remain substantially limited, with performance degrading as grounding and route complexity increase. We further present Grounding2Route as a strong benchmark method, which consistently improves hard-requirement satisfaction, soft-preference adherence, and path efficiency, with additional validation in real-world environments. Nevertheless, the large remaining gap to human performance indicates that Map2Route is far from saturated and provides substantial headroom for future research.

REFERENCES

- [1] C. Huang, O. Mees, A. Zeng, and W. Burgard, “Visual language maps for robot navigation,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 10 608–10 615.
- [2] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Suenderhauf, “Sayplan: Grounding large language models using 3d scene graphs for scalable robot task planning,” *arXiv preprint arXiv:2307.06135*, 2023.
- [3] Z. Dai, A. Asgharivaskasi, T. Duong, S. Lin, M.-E. Tzes, G. Pappas, and N. Atanasov, “Optimal scene graph planning with large language model guidance,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 14 062–14 069.
- [4] J. X. Liu, Z. Yang, I. Idrees, S. Liang, B. Schornstein, S. Tellex, and A. Shah, “Grounding complex natural language commands for temporal tasks in unseen environments,” in *Conference on Robot Learning*. Pmlr, 2023, pp. 1084–1110.
- [5] J. X. Liu, A. Shah, G. Konidaris, S. Tellex, and D. Paulius, “Lang2l1t-2: Grounding spatiotemporal navigation commands using large language and vision-language models,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 2325–2332.
- [6] B. Quartey, E. Rosen, S. Tellex, and G. Konidaris, “Verifiably following complex robot instructions with foundation models,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 1–8.
- [7] B. Rabiei, M. K. AR, Z. Dai, S. L. Pilla, Q. Dong, and N. Atanasov, “Ltlcodegen: Code generation of syntactically correct temporal logic for robot task planning,” in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2025, pp. 19 240–19 247.
- [8] K. Weerakoon, M. Elnoor, G. Seneviratne, V. Rajagopal, S. H. Arul, J. Liang, M. K. M. Jaffar, and D. Manocha, “Behav: Behavioral rule guided autonomy using vlms for robot navigation in outdoor scenes,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 7044–7051.
- [9] D. Huo, J. Wang, C. Gao, Y. Qiao, D. Zhang, and G. Zhou, “Norm-nav: Zero-shot mobile robot navigation with natural language behavioral constraints,” *arXiv preprint arXiv:2605.16979*, 2026.
- [10] P. Anderson, Q. Wu, D. Teney, J. Bruce, M. Johnson, N. Sünderhauf, I. Reid, S. Gould, and A. Van Den Hengel, “Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments,” in *2018 IEEE/CVF conference on computer vision and pattern recognition*. IEEE, 2018, pp. 3674–3683.
- [11] A. Ku, P. Anderson, R. Patel, E. Je, and J. Baldridge, “Room-across-room: Multilingual vision-and-language navigation with dense spatiotemporal grounding,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 4392–4412.
- [12] M. Bao, Y. Cai, H. Xu, Z. Li, J. He, J. Tang, C. Lv, J. Zhang, Y. Xie, and W. Wang, “Goal2pixel: Grounding goals to pixels for vision-language navigation,” *arXiv preprint arXiv:2606.01621*, 2026.
- [13] D. Batra, A. Gokaslan, A. Kembhavi, O. Maksymets, R. Mottaghi, M. Savva, A. Toshev, and E. Wijmans, “Objectnav revisited: On evaluation of embodied agents navigating to objects,” *arXiv preprint arXiv:2006.13171*, 2020.
- [14] Y. Cai, Z. Li, M. Wang, M. Bao, H. Zhu, R. Bai, D. Zhao, Z. Li, W. Wang, W.-Y. Yau *et al.*, “Intentnav: Learning spatial-visual object navigation from human demonstrations,” *arXiv preprint arXiv:2606.08029*, 2026.
- [15] Y. Qi, Q. Wu, P. Anderson, X. Wang, W. Y. Wang, C. Shen, and A. v. d. Hengel, “Reverie: Remote embodied visual referring expression in real indoor environments,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 9982–9991.
- [16] F. Zhu, X. Liang, Y. Zhu, Q. Yu, X. Chang, and X. Liang, “Soon: Scenario oriented object navigation with graph-based exploration,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 12 689–12 699.
- [17] M. Deitke, E. VanderBilt, A. Herrasti, L. Weihs, K. Ehsani, J. Salvador, W. Han, E. Kolve, A. Kembhavi, and R. Mottaghi, “Prothor: Large-scale embodied ai using procedural generation,” *Advances in neural information processing systems*, vol. 35, pp. 5982–5994, 2022.
- [18] E. Kolve, R. Mottaghi, W. Han, E. VanderBilt, L. Weihs, A. Herrasti, M. Deitke, K. Ehsani, D. Gordon, Y. Zhu *et al.*, “Ai2-thor: An interactive 3d environment for visual ai,” *arXiv preprint arXiv:1712.05474*, 2017.
- [19] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as policies: Language model programs for embodied control,” in *2023 IEEE International conference on robotics and automation (ICRA)*. IEEE, 2023, pp. 9493–9500.
- [20] H. Zhu, Z. Li, Z. Liu, K. Guo, Z. Lin, Y. Cai, G. Chen, C. Lv, W. Wang, J. Oh *et al.*, “Sysnav: Multi-level systematic cooperation enables real-world, cross-embodiment object navigation,” *IEEE Robotics and Automation Letters*, 2026.
- [21] J. Zhang, “Autonomy stack for mecanum wheel platform,” https://github.com/jizhang-cmu/autonomy_stack_mecanum_wheel_platform, 2024.
- [22] F. Xie, J. Zhang, and S. Schwertfeger, “Intelligent lidar navigation: Leveraging external information and semantic maps with llm as copilot,” in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2025, pp. 20 402–20 409.